Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería en Software

Análisis crítico comparativo de Facebook TAO y Amazon DynamoDB como sistemas distribuidos de almacenamiento masivo

Asignatura: Sistemas Distribuidos

Código de la actividad: TA-IND-01

Estudiante: Diego José Giraldo Cagua

Paralelo: 7mo A

Fecha: 13 de noviembre de 2026

Repositorio del proyecto

https://github.com/BelindaTZ/TA-IND-01-Informe-Acad-mico-Individual---CGMT.git

Resumen

Este informe estudia Facebook TAO como sistema distribuido de almacenamiento para el grafo social de Meta y lo compara con Amazon DynamoDB. El análisis considera las transparencias de distribución, los principios CAP y PACELC, los modelos de comunicación y la tolerancia a fallos. TAO organiza la información mediante objetos y asociaciones, mientras que DynamoDB ofrece un servicio NoSQL administrado basado en tablas y claves. La comparación permite observar que ambos sistemas buscan atender grandes volúmenes de solicitudes, aunque responden a necesidades diferentes. Para complementar la investigación se realizó una experimentación con DynamoDB Local, Docker y AWS CLI. Se creó una tabla, se insertó y consultó información, y se verificaron operaciones de modificación y eliminación. También se detuvo y reinició el contenedor. Después del reinicio, el servicio volvió a ejecutarse, pero la tabla creada anteriormente ya no estaba disponible. Este resultado corresponde a la configuración local utilizada y no representa una prueba del comportamiento del servicio administrado de AWS. Finalmente, se examinan las ventajas y limitaciones de cada propuesta y se explica por qué la elección de un sistema distribuido debe considerar el tipo de datos, las operaciones requeridas y las condiciones de disponibilidad y consistencia.

Palabras clave: sistemas distribuidos, Facebook TAO, Amazon DynamoDB, CAP, PACELC, replicación, tolerancia a fallos.

1. Introducción

Los sistemas distribuidos permiten que una aplicación utilice recursos de varios equipos conectados mediante una red. Esta organización facilita el crecimiento de los servicios y permite atender solicitudes desde diferentes ubicaciones. Sin embargo, también introduce problemas que no aparecen de la misma manera en un sistema centralizado. Entre ellos se encuentran los retrasos de comunicación, la pérdida de conexión, los fallos de nodos y las diferencias temporales entre las copias de los datos.

El diseño de estos sistemas requiere tomar decisiones relacionadas con la disponibilidad, la consistencia y la latencia. El teorema CAP explica las limitaciones que aparecen cuando existe una partición de red, mientras que PACELC considera también el compromiso entre latencia y consistencia cuando la comunicación funciona normalmente [1], [2].

Facebook TAO es un almacén distribuido desarrollado para atender las consultas del grafo social de Facebook. Su modelo representa entidades como usuarios y publicaciones mediante objetos, y relaciones como amistades o comentarios mediante asociaciones. Su diseño fue presentado por Bronson et al. como una solución que permite atender las consultas del grafo social mediante una infraestructura de almacenamiento y caché distribuida [3].

Amazon DynamoDB es un servicio de base de datos NoSQL administrado por Amazon Web Services. Permite almacenar y consultar información mediante tablas, elementos y claves. Su propuesta está orientada a aplicaciones que requieren escalabilidad y administración reducida de la infraestructura [4]. Aunque su nombre está relacionado con el sistema Dynamo presentado anteriormente por Amazon, ambos conceptos no deben tratarse como si fueran exactamente el mismo producto [5].

El sistema principal de este análisis es Facebook TAO y Amazon DynamoDB se utiliza como sistema de comparación. La elección de ambos permite estudiar dos propuestas que trabajan con almacenamiento distribuido, pero que responden a necesidades distintas. TAO está diseñado alrededor de las consultas del grafo social de Meta, mientras que DynamoDB proporciona un servicio de almacenamiento NoSQL de uso general.

La investigación se complementa con una experimentación propia en DynamoDB Local. Esta herramienta permite utilizar una API compatible con DynamoDB desde un entorno local. Las pruebas se realizaron con Docker y AWS CLI, sin utilizar una instancia de producción de Amazon Web Services.

El objetivo es analizar las decisiones de diseño de ambos sistemas y relacionarlas con los contenidos de la primera unidad de la asignatura. No se pretende afirmar que los dos productos sean equivalentes, sino identificar sus diferencias y explicar en qué situaciones cada enfoque puede resultar conveniente.

2. Arquitectura de los sistemas

2.1. Facebook TAO

TAO es un almacén distribuido diseñado para representar y consultar el grafo social de Facebook. Su estructura se basa en dos elementos principales: los objetos y las asociaciones. Los objetos representan entidades del sistema, mientras que las asociaciones representan las relaciones existentes entre ellas.

Por ejemplo, un usuario puede estar relacionado con una publicación, una fotografía o la cuenta de otra persona. Este modelo permite consultar las relaciones sin tener que construir

manualmente todas las operaciones sobre tablas relacionales desde cada aplicación.

La arquitectura de TAO utiliza una organización distribuida de cachés y almacenamiento persistente. En el diseño descrito por Bronson et al., las solicitudes pasan por niveles de servicio que permiten recuperar información desde las cachés antes de consultar la base de datos. Esto reduce la cantidad de accesos al almacenamiento persistente y permite atender un volumen elevado de lecturas [3].

El sistema también utiliza una organización regional para atender solicitudes y propagar cambios. La separación entre las operaciones de lectura y la propagación de las actualizaciones permite optimizar el acceso a información que se consulta con mucha frecuencia.

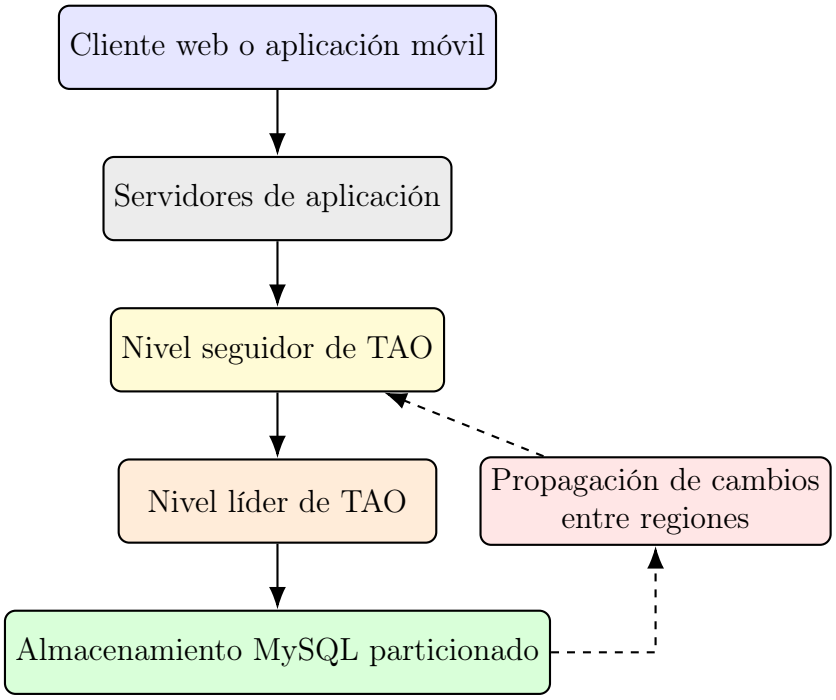


Figura 1: Representación simplificada de la arquitectura de Facebook TAO, elaborada a partir de la descripción de Bronson et al. [3].

La Figura 1 representa el recorrido general de una solicitud. No muestra todos los componentes internos de la infraestructura de Meta, sino una simplificación de los niveles de servicio, caché y almacenamiento descritos en la publicación original.

2.2. Amazon DynamoDB

DynamoDB organiza la información mediante tablas. Cada tabla contiene elementos y cada elemento está identificado por una clave primaria. Esta clave puede estar formada por una clave de partición o por una combinación de clave de partición y clave de ordenación.

La clave de partición permite distribuir los elementos entre las unidades internas de almacenamiento. La elección de esta clave es importante porque influye en la distribución de las solicitudes y en la posibilidad de evitar concentraciones excesivas de operaciones sobre una misma partición.

A diferencia de TAO, DynamoDB no presenta al desarrollador una estructura específica para representar un grafo social. El usuario define las tablas y las claves que necesita para

sus operaciones. Por este motivo, las relaciones entre entidades deben diseñarse de acuerdo con los patrones de acceso de la aplicación.

El servicio ofrece diferentes opciones de lectura. Las lecturas eventualmente consistentes pueden devolver información que todavía no refleja una escritura reciente, mientras que las lecturas fuertemente consistentes permiten solicitar el dato más actualizado disponible según las garantías documentadas por AWS [6].

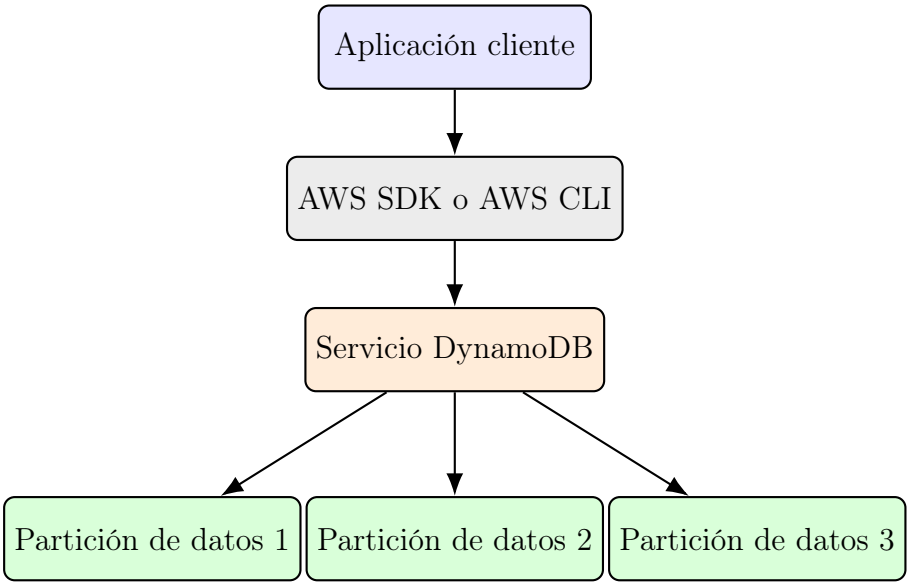


Figura 2: Representación general del acceso a Amazon DynamoDB.

La Figura 2 representa la interacción desde la aplicación hasta el almacenamiento. Es una representación conceptual y no pretende mostrar todos los componentes internos del servicio.

2.3. Diferencias entre las arquitecturas

La diferencia principal se encuentra en el modelo de información. TAO proporciona una estructura orientada a objetos y asociaciones, adecuada para representar las relaciones de una red social. DynamoDB permite definir tablas y claves según las necesidades de cada aplicación.

También existe una diferencia en la forma de utilizar los sistemas. TAO forma parte de la infraestructura interna de Meta y está adaptado a sus aplicaciones. DynamoDB se ofrece como un servicio administrado, por lo que una organización puede utilizarlo sin construir su propia infraestructura de almacenamiento distribuido.

El diseño de TAO está relacionado con un patrón de consultas concreto. DynamoDB permite utilizar diferentes estructuras de datos, pero exige que el desarrollador diseñe las claves y los patrones de acceso antes de implementar las operaciones.

2.4. Evolución desde el esquema inicial

Para elaborar los diagramas digitales se tomó como referencia el esquema inicial realizado a mano, que se incluye en el Anexo A. En el bosquejo se identificaron los componentes principales de los dos sistemas y la relación general entre ellos.

Posteriormente se organizaron los componentes en niveles para representar el recorrido de las solicitudes. En el caso de TAO se separaron los servidores de aplicación, los niveles de servicio y el almacenamiento. Para DynamoDB se representó el acceso desde el cliente y la distribución conceptual de los datos.

Esta modificación permitió diferenciar el modelo de grafo social de TAO y el modelo de tablas y claves de DynamoDB.

3. Transparencias de distribución

Las transparencias de distribución permiten que una aplicación utilice recursos distribuidos sin tener que administrar todos sus detalles físicos. Entre las transparencias que se analizan se encuentran acceso, ubicación, migración, reubicación, replicación, concurrencia y fallos.

La siguiente tabla resume la interpretación realizada a partir de las características documentadas de ambos sistemas.

Tabla 1: Comparación de las transparencias de distribución.

Transparencia	**Facebook TAO**	**Amazon DynamoDB**
Acceso	La API permite consultar objetos y asociaciones sin acceder directamente a la base de datos.	La aplicación utiliza operaciones de la API sin administrar directamente el almacenamiento.
Ubicación	La aplicación utiliza los servicios de TAO sin seleccionar directamente cada servidor físico.	El cliente utiliza el servicio mediante un endpoint regional.
Migración	La organización de los datos depende de la distribución interna y de los fragmentos utilizados.	El servicio administra la distribución de los datos sin que el usuario gestione directamente los servidores.
Reubicación	La información pública consultada no permite determinar todas las garantías de reubicación interna.	La ubicación física de los datos no es administrada directamente por el cliente.
Replicación	Utiliza mecanismos de caché y propagación de cambios entre regiones.	Dispone de opciones de replicación, incluidas las tablas globales.
Concurrencia	Su diseño debe considerar que las copias pueden presentar diferencias temporales.	Permite elegir entre lecturas eventualmente consistentes y fuertemente consistentes.
Fallos	La infraestructura puede ocultar determinados fallos internos, pero depende de la disponibilidad de la red.	El servicio administra la infraestructura, aunque pueden presentarse interrupciones de acceso.

3.1. Transparencias en Facebook TAO

La transparencia de acceso se observa en la API que permite trabajar con objetos y asociaciones. Las aplicaciones no necesitan realizar directamente las operaciones sobre cada

componente de almacenamiento. Esta separación facilita el desarrollo de funcionalidades que consultan relaciones dentro del grafo social [3].

La transparencia de ubicación se presenta porque las aplicaciones utilizan los servicios de TAO sin tener que seleccionar manualmente el servidor donde se encuentra cada objeto. Sin embargo, esto no significa que los desarrolladores desconozcan por completo la organización física de los datos.

La replicación es importante para mantener copias de información y atender consultas desde diferentes regiones. No obstante, la propagación de los cambios puede producir diferencias temporales entre las copias. Por esta razón, las aplicaciones deben considerar que una lectura no siempre representa inmediatamente la última actualización.

En cuanto a la transparencia de concurrencia, el sistema debe controlar las operaciones simultáneas sobre objetos y asociaciones. La documentación original describe los mecanismos utilizados por TAO para gestionar estas operaciones y mantener el funcionamiento de la infraestructura [3].

La transparencia de fallos es parcial. La infraestructura puede gestionar ciertos problemas internos, pero una interrupción de red que afecte el acceso a los centros de datos puede impedir que los usuarios lleguen al servicio. El incidente de Meta del 4 de octubre de 2021 permite observar esta limitación [7].

3.2. Transparencias en Amazon DynamoDB

DynamoDB ofrece transparencia de acceso mediante una API que permite realizar operaciones sobre tablas y elementos. El usuario no necesita administrar directamente los servidores donde se almacena la información.

La transparencia de ubicación también está presente porque el cliente utiliza un endpoint regional y no selecciona directamente la ubicación física de cada elemento.

La replicación puede utilizarse para mantener copias de los datos y distribuirlos entre regiones mediante tablas globales. Esta característica permite diseñar aplicaciones que necesitan consultar información desde diferentes ubicaciones geográficas [8].

La concurrencia requiere considerar el tipo de lectura utilizado. DynamoDB permite solicitar lecturas eventualmente consistentes o fuertemente consistentes, según las necesidades de la aplicación [6].

La transparencia de fallos depende de los mecanismos administrados por AWS. El usuario no gestiona directamente los servidores, pero puede percibir una interrupción cuando el endpoint o los servicios de comunicación no están disponibles.

4. CAP y PACELC

El teorema CAP describe tres propiedades importantes de los sistemas distribuidos: consistencia, disponibilidad y tolerancia a particiones. Cuando existe una partición de red, el sistema debe afrontar un compromiso entre mantener la consistencia y continuar respondiendo a las solicitudes [1].

PACELC amplía esta explicación. Durante una partición, considera el compromiso entre disponibilidad y consistencia. Cuando no existe una partición, considera el compromiso entre latencia y consistencia [2].

Estas propiedades no deben utilizarse como etiquetas absolutas sin considerar el tipo de operación. Un mismo servicio puede ofrecer diferentes garantías dependiendo de la solicitud y de las condiciones en las que se ejecuta.

Tabla 2: Interpretación de CAP y PACELC en ambos sistemas.

Aspecto	**Facebook TAO**	**Amazon DynamoDB**
Consistencia	El diseño considera la propagación de cambios y las diferencias temporales entre copias.	Permite lecturas eventualmente consistentes y fuertemente consistentes.
Disponibilidad	Las cachés y la organización distribuida ayudan a atender consultas.	El servicio administra la infraestructura y ofrece mecanismos de disponibilidad.
Partición de red	Una interrupción de comunicación puede afectar el acceso entre componentes o regiones.	La disponibilidad de las operaciones depende de la conectividad y de las garantías de cada lectura.
Latencia	El uso de cachés reduce la necesidad de consultar siempre el almacenamiento persistente.	El cliente puede elegir el tipo de lectura según sus necesidades.
Interpretación PACELC	Se prioriza un acceso eficiente a las consultas del grafo, aceptando diferencias temporales entre copias.	El desarrollador puede elegir entre lecturas eventuales y fuertes, considerando sus efectos.

4.1. Facebook TAO

TAO está orientado a atender una gran cantidad de consultas sobre el grafo social. Para conseguirlo, utiliza mecanismos de almacenamiento y caché que reducen la necesidad de consultar directamente la base de datos en cada solicitud [3].

Esta organización permite mejorar el tiempo de respuesta, pero también exige considerar la propagación de las actualizaciones. Cuando existen varias copias de la información, una modificación puede tardar en reflejarse en todas ellas.

Desde la perspectiva de PACELC, el diseño de TAO muestra una preferencia por atender consultas de forma eficiente, incluso cuando las copias pueden presentar diferencias temporales. Esta interpretación se relaciona con su arquitectura y con el uso de cachés, pero no debe entenderse como una clasificación oficial publicada por Meta.

4.2. Amazon DynamoDB

DynamoDB ofrece diferentes alternativas de consistencia para las operaciones de lectura. Las lecturas eventualmente consistentes pueden devolver información que todavía no refleja una escritura reciente. Las lecturas fuertemente consistentes permiten solicitar información más actualizada, aunque la aplicación debe considerar las condiciones y limitaciones documentadas por AWS [6].

Esta posibilidad permite adaptar el comportamiento de las consultas a las necesidades de cada aplicación. Por ejemplo, una consulta que muestra información general puede aceptar una actualización con cierto retraso, mientras que una operación que requiere comprobar el estado más reciente puede utilizar una lectura fuerte.

Las tablas globales permiten distribuir información entre regiones. Sin embargo, el uso de varias regiones también requiere considerar la propagación de los cambios y el comportamiento de las aplicaciones ante interrupciones de comunicación [8].

Por tanto, no resulta adecuado clasificar todas las operaciones de DynamoDB con una única etiqueta CAP sin distinguir el tipo de lectura y las condiciones de funcionamiento.

5. Modelos de comunicación

Los sistemas distribuidos utilizan mecanismos de comunicación para intercambiar solicitudes y respuestas entre sus componentes. La elección de estos mecanismos influye en la latencia, la organización de los servicios y la manera en que se propagan los cambios.

5.1. Facebook TAO

TAO utiliza una organización cliente-servidor distribuida. Las aplicaciones envían solicitudes a los servicios de TAO y estos consultan los niveles de caché y almacenamiento correspondientes.

Desde el punto de vista de la aplicación, una consulta se realiza mediante una operación solicitada al servicio. Internamente, la infraestructura debe coordinar el acceso a las cachés y a la base de datos.

La propagación de los cambios entre componentes permite mantener las copias utilizadas por las diferentes regiones. Esta organización es necesaria porque una actualización realizada en un punto del sistema debe reflejarse posteriormente en las demás copias que participan en las consultas [3].

5.2. Amazon DynamoDB

DynamoDB utiliza un modelo de comunicación cliente-servicio. La aplicación realiza solicitudes mediante herramientas como AWS SDK o AWS CLI y recibe una respuesta con el resultado de la operación.

El cliente no necesita comunicarse directamente con los servidores internos donde se encuentran los datos. La infraestructura administrada por AWS se encarga de procesar las solicitudes y acceder a la información correspondiente.

La documentación de AWS también describe opciones de consistencia y replicación que influyen en el comportamiento de las operaciones [6], [8].

5.3. Comparación

Ambos sistemas utilizan una organización de tipo cliente-servidor, pero su propósito y nivel de exposición son diferentes.

TAO es una infraestructura interna desarrollada para las necesidades de Meta. Sus componentes están organizados para atender las consultas del grafo social.

DynamoDB ofrece una interfaz pública para que diferentes aplicaciones puedan utilizar el almacenamiento sin administrar directamente su infraestructura. En este caso, el desarrollador trabaja principalmente con tablas, claves y operaciones de la API.

La diferencia no se limita al protocolo utilizado. También está relacionada con la forma en que cada sistema organiza los datos y con la responsabilidad que asume el equipo que desarrolla la aplicación.

6. Tolerancia a fallos

La tolerancia a fallos permite que un sistema continúe funcionando o recupere sus operaciones cuando alguno de sus componentes presenta problemas. En un entorno distribuido, los fallos pueden producirse en los servidores, en el almacenamiento o en los mecanismos de comunicación.

6.1. Facebook TAO

TAO utiliza una arquitectura distribuida con diferentes niveles de servicio y almacenamiento. La utilización de cachés permite atender determinadas consultas sin depender de un acceso directo a la base de datos en cada operación.

Sin embargo, la existencia de varias copias también introduce la necesidad de mantener la información actualizada. Cuando se modifica un objeto o una asociación, los mecanismos de propagación deben comunicar el cambio a los componentes que mantienen copias de los datos [3].

Un ejemplo de los límites de la disponibilidad de una infraestructura distribuida fue el incidente de Meta del 4 de octubre de 2021. Según el informe publicado por la empresa, un cambio en la configuración de la red afectó la comunicación entre los centros de datos y provocó que los servicios dejaran de estar disponibles desde Internet [7].

Este caso permite observar que la replicación de datos no garantiza por sí sola la disponibilidad completa de una aplicación. Si los usuarios no pueden llegar a los servicios debido a problemas de red o enrutamiento, las copias de información no son suficientes para mantener el acceso.

6.2. Amazon DynamoDB

DynamoDB es un servicio administrado. Por esta razón, el usuario no necesita gestionar directamente los servidores que mantienen la información. AWS se encarga de la infraestructura y proporciona mecanismos de disponibilidad y replicación.

Las tablas globales permiten distribuir información entre regiones. Esta característica puede utilizarse para diseñar aplicaciones que necesiten mantener datos disponibles en distintas ubicaciones geográficas [8].

La tolerancia a fallos no significa que el servicio nunca pueda presentar interrupciones. Una aplicación debe considerar errores de comunicación, tiempos de espera y la posibilidad de que un endpoint no responda temporalmente.

Por esta razón, la disponibilidad del almacenamiento debe analizarse junto con la disponibilidad de la red y de los servicios que permiten acceder a los datos.

6.3. Comparación de los mecanismos

TAO utiliza una infraestructura adaptada a las necesidades internas de Meta, mientras que DynamoDB ofrece una infraestructura administrada por AWS.

En TAO, el diseño de las cachés y la propagación de cambios forman parte de la arquitectura que permite atender las consultas. En DynamoDB, la administración de la infraestructura queda principalmente a cargo del proveedor.

En ambos casos, la tolerancia a fallos debe considerar algo más que la conservación de los datos. También debe contemplar si los clientes pueden comunicarse con el servicio y si las aplicaciones pueden recuperarse después de una interrupción.

7. Experimentación propia

7.1. Objetivo

La experimentación tuvo como objetivo comprobar el funcionamiento básico de DynamoDB Local y observar qué ocurre cuando se interrumpe y se reinicia el contenedor que ejecuta el servicio.

Las pruebas se realizaron el 13 de noviembre de 2026 utilizando Windows 11, PowerShell, Docker Desktop y AWS CLI. Se utilizó DynamoDB Local porque permite ejecutar operaciones compatibles con la API de DynamoDB desde un entorno local.

Es importante señalar que DynamoDB Local no reproduce toda la infraestructura distribuida del servicio administrado de AWS. En particular, la prueba se realizó con un único contenedor, por lo que no se evaluó la replicación entre regiones o zonas de disponibilidad.

7.2. Preparación del entorno

Primero se comprobó la disponibilidad de Docker y AWS CLI.

```
docker --version
docker ps
aws --version
```

Después se descargó y ejecutó la imagen de DynamoDB Local.

```
docker pull amazon/dynamodb-local

docker run -d -p 8000:8000 --name dynamodb-local amazon/dynamodb-local
```

Se configuraron las credenciales locales y la región para que AWS CLI pudiera realizar las solicitudes.

```
$env:AWS_ACCESS_KEY_ID="local"
$env:AWS_SECRET_ACCESS_KEY="local"
$env:AWS_DEFAULT_REGION="us-east-1"
```

La conexión se comprobó mediante el siguiente comando:

```
aws dynamodb list-tables --endpoint-url http://localhost:8000
```

7.3. Creación de la tabla

Se creó una tabla denominada `Vehiculos`. La clave de partición fue el campo `placa`, de tipo cadena.

```
aws dynamodb create-table --table-name Vehiculos ‘
  --attribute-definitions AttributeName=placa,AttributeType=S ‘
  --key-schema AttributeName=placa,KeyType=HASH ‘
  --billing-mode PAY_PER_REQUEST ‘
  --endpoint-url http://localhost:8000
```

La operación devolvió la información de la tabla y permitió comprobar que su estado era `ACTIVE`.

Archivo pendiente: `capturas/captura`$_{1c}reacion_{t}abla.png$

Figura 3: Creación de la tabla Vehiculos en DynamoDB Local.

7.4. Inserción y consulta de información

Para comprobar las operaciones de escritura y lectura se utilizó un archivo JSON con los datos de un vehículo.

El archivo `vehiculo1.json` contenía la siguiente información:

```
{
  "placa": {"S": "PBC-2045"},
  "marca": {"S": "Chevrolet"},
  "anio": {"N": "2018"},
  "aprobado": {"BOOL": false}
}
```

El registro se insertó mediante el comando `put-item`.

```
aws dynamodb put-item --table-name Vehiculos ‘
  --item file://experimento/scripts/vehiculo1.json ‘
  --endpoint-url http://localhost:8000
```

A continuación, se consultó la información almacenada.

```
aws dynamodb scan --table-name Vehiculos ‘
  --endpoint-url http://localhost:8000
```

La consulta devolvió un registro. El resultado mostró `Count: 1` y `ScannedCount: 1`, lo que confirmó que la información se había insertado y podía recuperarse.

Archivo pendiente: `capturas/captura`$_{2i}nsercion_{c}onsulta.png$

Figura 4: Inserción y consulta del registro de prueba.

7.5. Actualización y eliminación

También se realizaron operaciones de modificación y eliminación para comprobar el comportamiento básico de la tabla.

La actualización permitió modificar el valor de un atributo del registro mediante una expresión de actualización. Para evitar problemas con el formato JSON de PowerShell, los parámetros se prepararon en archivos independientes y se enviaron mediante la opción `file://`.

La eliminación se realizó utilizando la clave de partición del elemento que se quería retirar.

Estas operaciones permitieron comprobar que DynamoDB Local aceptaba las solicitudes de modificación y eliminación sobre los elementos de la tabla. Los comandos y archivos utilizados se encuentran organizados en la carpeta `experimento/scripts` del repositorio.

7.6. Interrupción del servicio

Para observar el comportamiento del servicio ante una interrupción, se detuvo el contenedor mediante el siguiente comando:

```
docker stop dynamodb-local
```

Mientras el contenedor estuvo detenido, se intentó consultar la lista de tablas.

```
aws dynamodb list-tables --endpoint-url http://localhost:8000
```

La solicitud devolvió un error de conexión con el endpoint `http://localhost:8000`. Esto confirmó que el servicio no estaba disponible mientras el contenedor permanecía detenido.

Archivo pendiente: `capturas/captura`$_{3i}$*nterrupcion.png*

Figura 5: Error de conexión al detener el contenedor.

7.7. Reinicio del contenedor

Después se inició nuevamente el contenedor y se esperó unos segundos para permitir que el servicio estuviera disponible.

```
docker start dynamodb-local

Start-Sleep -Seconds 5

docker ps --filter "name=dynamodb-local"
```

El contenedor volvió a aparecer en ejecución y el puerto 8000 quedó publicado.

A continuación, se consultó nuevamente la lista de tablas.

```
aws dynamodb list-tables --endpoint-url http://localhost:8000
```

El resultado fue una lista vacía:

```
{
    "TableNames": []
}
```

Finalmente, se intentó consultar la tabla que se había creado antes de detener el contenedor.

```
aws dynamodb scan --table-name Vehiculos ‘
  --endpoint-url http://localhost:8000
```

La operación devolvió el error `ResourceNotFoundException`, indicando que la tabla `Vehiculos` no existía.

Archivo pendiente: `capturas/captura`$_{4r}$*einicio.png*

Figura 6: Reinicio del contenedor y comprobación de la ausencia de la tabla.

7.8. Interpretación de los resultados

La prueba permitió observar dos situaciones diferentes. Mientras el contenedor estaba detenido, el servicio no respondió a las solicitudes realizadas desde AWS CLI. Después de iniciar nuevamente el contenedor, el endpoint volvió a estar disponible, pero la tabla creada anteriormente ya no aparecía.

Por lo tanto, la recuperación del proceso no significó que también se recuperara la información almacenada. El contenedor volvió a ejecutarse, pero el estado anterior no estaba disponible.

La causa exacta de la ausencia de la tabla no se determinó durante esta prueba. Una posible explicación es que la configuración utilizada no estaba conservando los datos mediante un directorio persistente. Para comprobarlo sería necesario revisar la configuración de almacenamiento de DynamoDB Local y repetir el experimento utilizando un directorio de datos montado como volumen.

Este resultado no demuestra que DynamoDB en AWS pierda los datos cuando se reinicia un componente. La prueba se realizó con una instancia local de un solo contenedor y no incluyó los mecanismos de replicación y disponibilidad del servicio administrado.

Tampoco permite comprobar el teorema CAP, porque no se provocó una partición de red entre réplicas. Lo que se observó fue la interrupción completa de un servicio local y la ausencia de la tabla después del reinicio.

La principal conclusión experimental es que la disponibilidad del proceso y la persistencia de los datos son aspectos diferentes. Un servicio puede volver a ejecutarse y, aun así, no recuperar el estado que tenía antes de una interrupción.

8. Ejemplos del entorno del estudiante

8.1. Consulta de información en el SGA de la UTEQ

Un ejemplo relacionado con los sistemas distribuidos es el uso del Sistema de Gestión Académica de la Universidad Técnica Estatal de Quevedo. Cuando un estudiante consulta sus notas, horarios o información académica, realiza una solicitud a una aplicación que debe recuperar los datos correspondientes.

Desde el punto de vista del usuario, la operación parece sencilla. Sin embargo, detrás de ella intervienen componentes de aplicación, comunicación y almacenamiento.

Si el servicio presenta problemas de conexión o deja de responder, el estudiante puede percibir directamente la interrupción. Esto permite relacionar el uso cotidiano del SGA con la transparencia de fallos: el usuario no necesita conocer qué componente presentó el problema para notar que la operación no pudo completarse.

8.2. Uso de una aplicación bancaria

Otro ejemplo se puede observar en las aplicaciones bancarias. Cuando una persona realiza una operación, la aplicación envía una solicitud al servicio correspondiente y posteriormente muestra una respuesta.

En algunas situaciones, la confirmación de una operación y la actualización de la información que aparece en pantalla pueden no producirse exactamente al mismo tiempo. Este comportamiento permite comprender por qué la consistencia es importante en los sistemas distribuidos.

En una operación financiera, la aplicación debe considerar cuidadosamente qué información puede mostrarse con cierto retraso y qué operaciones necesitan consultar el estado más actualizado. Esta diferencia se relaciona con las opciones de consistencia que ofrece DynamoDB [6].

9. Análisis comparativo

La comparación entre Facebook TAO y Amazon DynamoDB permite identificar diferencias en el modelo de datos, la administración de la infraestructura y las operaciones que cada sistema busca atender.

Tabla 3: Comparación general entre Facebook TAO y Amazon DynamoDB.

Criterio	**Facebook TAO**	**Amazon DynamoDB**
Propósito	Almacenar y consultar el grafo social de Meta.	Ofrecer almacenamiento NoSQL administrado para diferentes aplicaciones.
Modelo de datos	Objetos y asociaciones.	Tablas, elementos y claves.
Acceso	Mediante servicios internos de TAO.	Mediante una API pública y herramientas como AWS SDK y AWS CLI.
Organización	Infraestructura distribuida con niveles de caché y almacenamiento.	Servicio administrado que distribuye la información internamente.
Consistencia	Considera la propagación de cambios entre copias.	Permite lecturas eventualmente consistentes y fuertemente consistentes.
Replicación	Propagación de cambios y organización de copias entre regiones.	Replicación administrada y tablas globales.
Comunicación	Servicios internos que atienden las consultas del grafo.	Solicitudes de clientes mediante la API del servicio.
Administración	Infraestructura interna de Meta.	Infraestructura administrada por AWS.
Acceso público	No se ofrece como un servicio general para cualquier desarrollador.	Se ofrece como servicio comercial de AWS.

Una diferencia importante está en la forma de representar la información. TAO utiliza objetos y asociaciones porque su propósito está relacionado con las consultas del grafo social. DynamoDB utiliza tablas y claves, lo que permite adaptar la estructura de almacenamiento a diferentes aplicaciones.

En cuanto a la administración, TAO requiere una infraestructura desarrollada y mantenida por Meta. DynamoDB permite que las organizaciones utilicen almacenamiento distribuido sin tener que administrar directamente los servidores.

La consistencia también representa una diferencia importante. TAO está diseñado alrededor de las necesidades de las aplicaciones de Meta y del comportamiento de su infraestructura. DynamoDB permite seleccionar el tipo de lectura según las necesidades de cada operación [6].

Ambos sistemas necesitan mecanismos para mantener la información disponible y gestionar los cambios. Sin embargo, sus decisiones de diseño responden a patrones de uso distintos.

10. Postura crítica

Facebook TAO es una solución adecuada para una plataforma que necesita consultar una gran cantidad de relaciones entre usuarios, publicaciones y otros elementos de un grafo social. Su arquitectura está relacionada directamente con las necesidades de Meta, por

lo que no sería correcto considerarlo una base de datos genérica que pueda utilizarse sin modificaciones en cualquier organización.

Una de sus ventajas es que ofrece un modelo de información cercano a las operaciones que necesita una red social. Las aplicaciones pueden trabajar con objetos y asociaciones sin tener que construir desde cero todos los mecanismos de acceso y almacenamiento [3].

Su principal limitación para una organización externa es que no se ofrece como un servicio público administrado. Además, implementar una infraestructura con características similares requiere conocimientos y recursos importantes.

DynamoDB presenta una propuesta diferente. Su administración está a cargo de AWS y el desarrollador puede concentrarse en las tablas, las claves y las operaciones de la aplicación. También permite elegir entre lecturas eventualmente consistentes y fuertemente consistentes, según las necesidades del sistema [6].

Sin embargo, DynamoDB exige diseñar correctamente las claves y los patrones de acceso. Una estructura de tablas que no corresponda a las consultas de la aplicación puede dificultar el desarrollo o afectar el rendimiento. También debe considerarse la dependencia del proveedor, ya que el servicio utiliza una API y una infraestructura propias de AWS.

La experimentación local permitió comprobar las operaciones básicas de almacenamiento y observar el comportamiento de una instancia cuando se interrumpe y se reinicia. No obstante, no sería correcto utilizar el resultado de esa prueba para afirmar que el servicio administrado de AWS presenta la misma limitación. La instancia local utilizada no contaba con una infraestructura distribuida comparable.

Desde mi punto de vista, la elección entre ambos sistemas depende principalmente del problema que se desea resolver. TAO tiene sentido en una plataforma que necesita administrar un grafo social de gran tamaño y cuenta con los recursos para mantener su infraestructura. DynamoDB resulta más accesible para organizaciones que necesitan almacenamiento NoSQL administrado y prefieren no construir su propia infraestructura distribuida.

No existe una solución que sea superior en todos los escenarios. La decisión debe considerar el modelo de datos, las operaciones más frecuentes, la consistencia requerida, la latencia aceptable y los recursos disponibles para administrar el servicio.

11. Conclusiones

Facebook TAO y Amazon DynamoDB permiten estudiar dos formas diferentes de abordar el almacenamiento distribuido. TAO está diseñado para administrar el grafo social de Meta mediante objetos, asociaciones y una infraestructura de cachés. DynamoDB ofrece un servicio NoSQL administrado que utiliza tablas y claves para almacenar información de diferentes aplicaciones.

El análisis de las transparencias de distribución permitió observar que ambos sistemas ocultan parte de la complejidad de su infraestructura. Sin embargo, esa transparencia no elimina todos los problemas relacionados con la ubicación de los datos, la comunicación, la concurrencia y los fallos.

CAP y PACELC ayudan a comprender que la disponibilidad, la consistencia y la latencia deben analizarse según las condiciones de funcionamiento y el tipo de operación. DynamoDB ofrece diferentes opciones de lectura, mientras que TAO responde a las necesidades particulares de las consultas del grafo social.

La experimentación con DynamoDB Local permitió comprobar la creación de una tabla, la inserción, consulta, modificación y eliminación de información. También permitió

observar que detener el contenedor interrumpe el acceso al servicio. Después del reinicio, el proceso volvió a ejecutarse, pero la tabla anterior no estaba disponible.

Este último resultado demuestra que la recuperación de un proceso no garantiza por sí sola la recuperación de los datos. Sin embargo, no representa el funcionamiento de DynamoDB en AWS, ya que la prueba se realizó en una instancia local de un solo contenedor.

Finalmente, se concluye que la selección de un sistema distribuido debe partir de las necesidades reales de la aplicación. El modelo de datos, las operaciones requeridas, la consistencia y la capacidad de recuperación son factores que deben evaluarse antes de elegir una tecnología.

12. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT como herramienta de apoyo durante el desarrollo del trabajo. Su uso estuvo relacionado con la organización de algunos apartados, la revisión de la redacción y la preparación de comandos para la experimentación con DynamoDB Local.

Las pruebas descritas en el informe fueron ejecutadas en el equipo del estudiante mediante PowerShell, Docker y AWS CLI. Las capturas y los resultados se obtuvieron de esas ejecuciones. La interpretación de los resultados se realizó considerando las limitaciones del entorno local y las diferencias con el servicio administrado de AWS.

Las fuentes bibliográficas se presentan en el archivo `referencias.bib`. Las afirmaciones técnicas deben contrastarse con las publicaciones y la documentación correspondientes antes de la entrega.

Referencias

[1] E. Brewer, «CAP Twelve Years Later: How the "Rules" Have Changed,» *Computer*, vol. 45, n.º 2, págs. 23-29, 2012. DOI: 10.1109/MC.2012.37 dirección: https://doi.org/10.1109/MC.2012.37

[2] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: 10.1109/MC.2012.33 dirección: https://doi.org/10.1109/MC.2012.33

[3] N. Bronson et al., «TAO: Facebook's Distributed Data Store for the Social Graph,» en *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, San Jose, CA, USA: USENIX Association, 2013, págs. 49-60, ISBN: 978-1-931971-01-0. dirección: https://www.usenix.org/conference/atc13/technical-sessions/presentation/bronson

[4] S. Sivasubramanian, «Amazon DynamoDB: A Seamlessly Scalable Non-relational Database Service,» en *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD '12)*, New York, NY, USA: ACM, 2012, págs. 729-730. DOI: 10.1145/2213836.2213945 dirección: https://doi.org/10.1145/2213836.2213945

[5] G. DeCandia et al., «Dynamo: Amazon's Highly Available Key-value Store,» en *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*, New York, NY, USA: ACM, 2007, págs. 205-220. DOI: 10.1145/1294261.1294281 dirección: https://doi.org/10.1145/1294261.1294281

[6] Amazon Web Services, *DynamoDB Read Consistency*, Amazon DynamoDB Developer Guide, 2026. dirección: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

[7] Meta Engineering, *More Details About the October 4 Outage*, Engineering at Meta, oct. de 2021. dirección: https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

[8] Amazon Web Services, *Amazon DynamoDB Global Tables*, AWS Documentation, 2026. dirección: https://aws.amazon.com/dynamodb/global-tables/

A. Anexo A: Esquema inicial realizado a mano

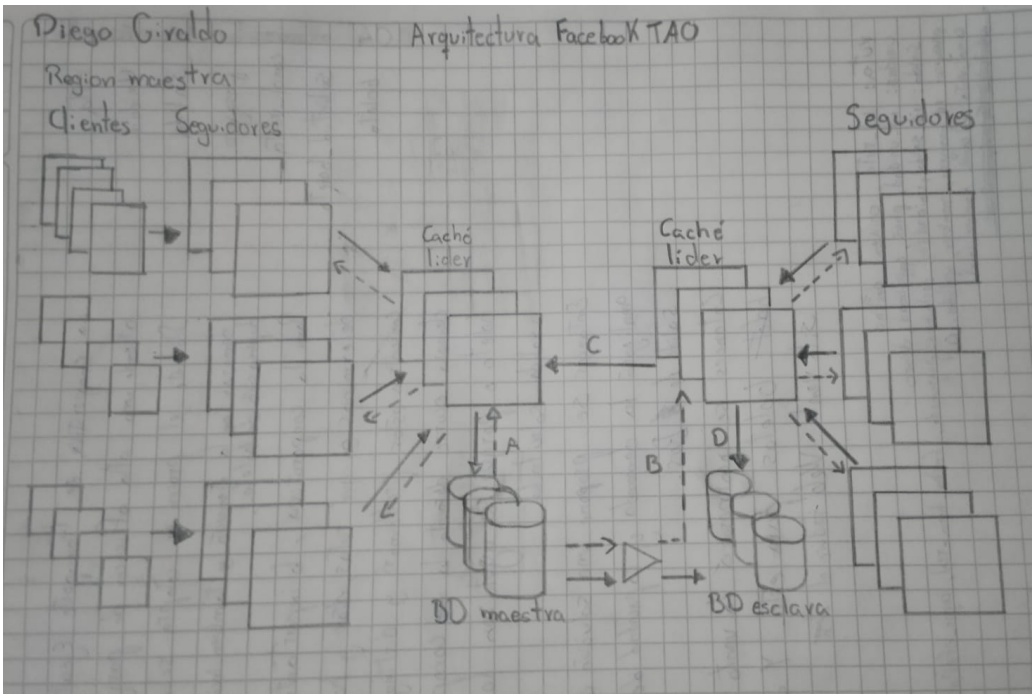


Figura 7: Esquema inicial a mano de Facebook TAO y Amazon DynamoDB.

B. Anexo B: Lectura y revisión de fuentes

Las siguientes evidencias corresponden a la revisión de las publicaciones y documentación utilizadas para desarrollar el análisis.

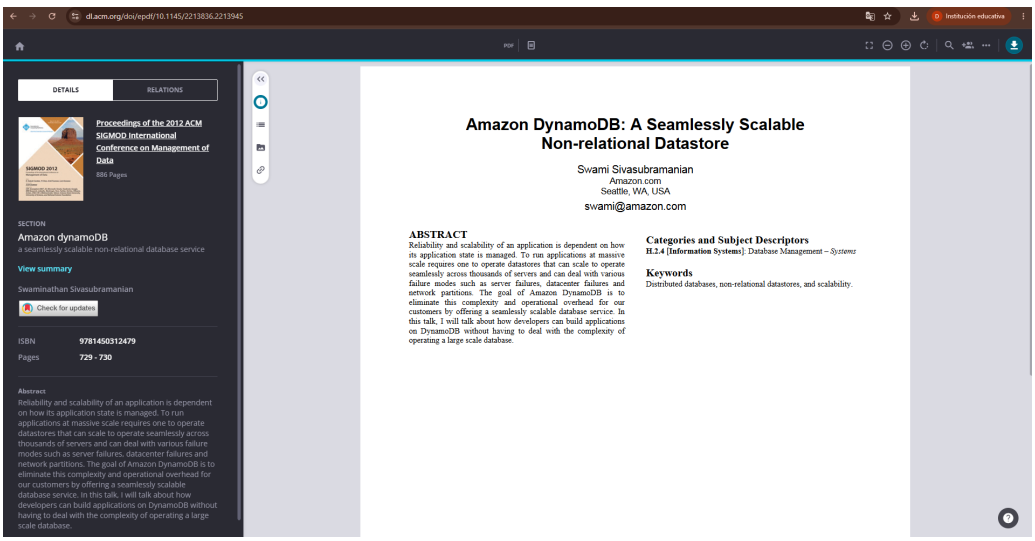


Figura 8: Lectura anotada del artículo original de Facebook TAO.

C. Anexo C: Evidencias de la experimentación

Las capturas de la experimentación se encuentran organizadas en la carpeta `capturas` del repositorio. En ellas se muestran la creación de la tabla, la inserción y consulta de información, la interrupción del servicio y el reinicio del contenedor.

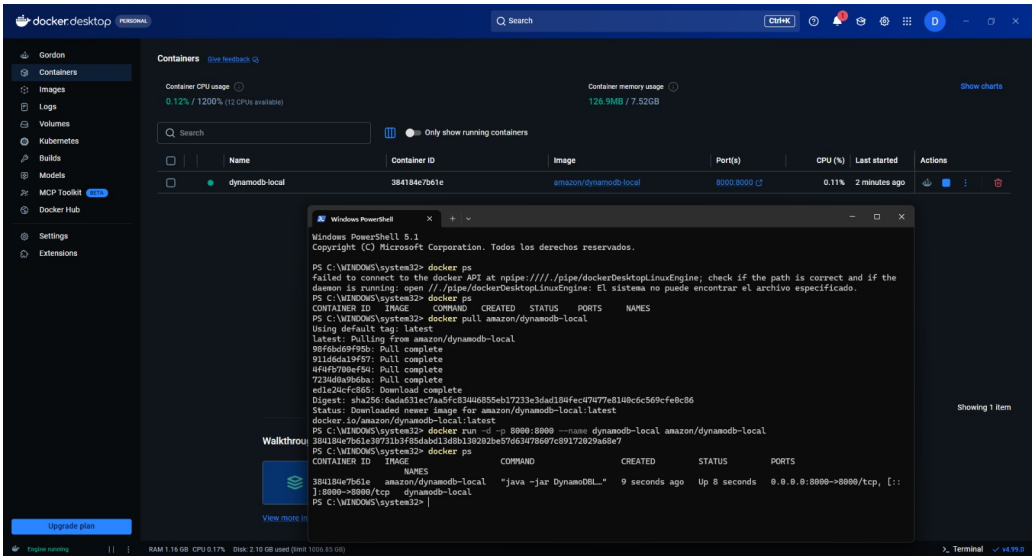


Figura 9: Ejecución de DynamoDB de forma local.

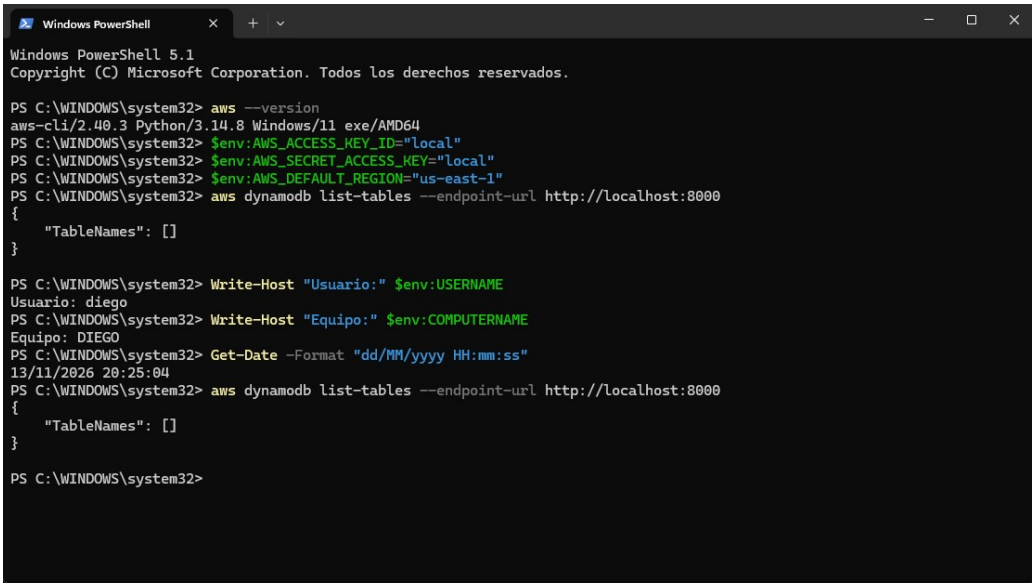


Figura 10: Conexión con DynamoDB.

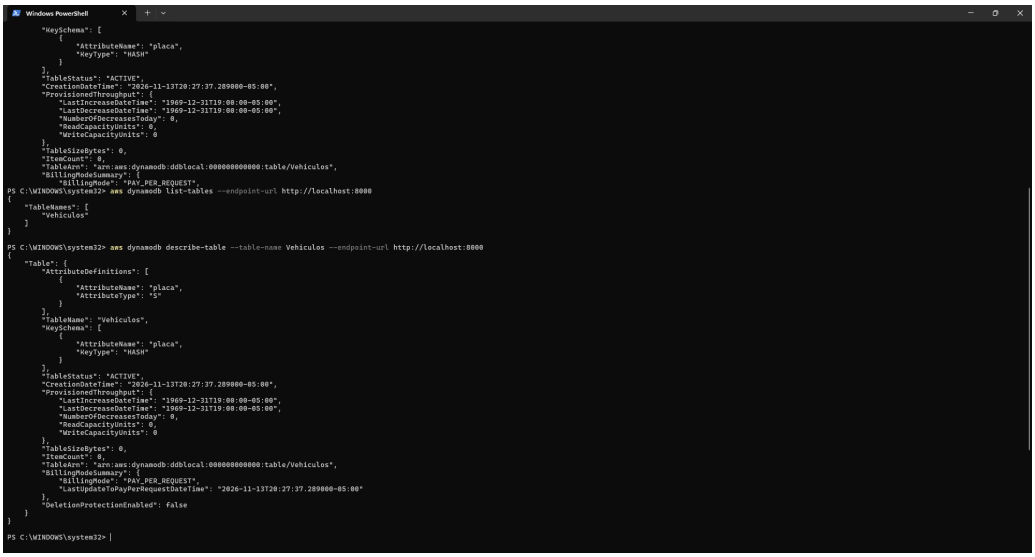


Figura 11: Creación de la primera tabla

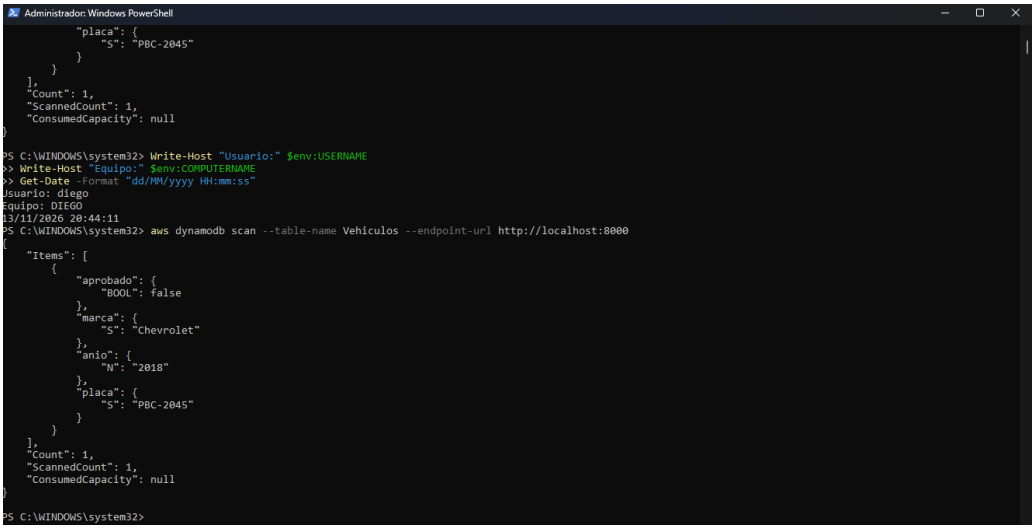


Figura 12: Inserción de registros

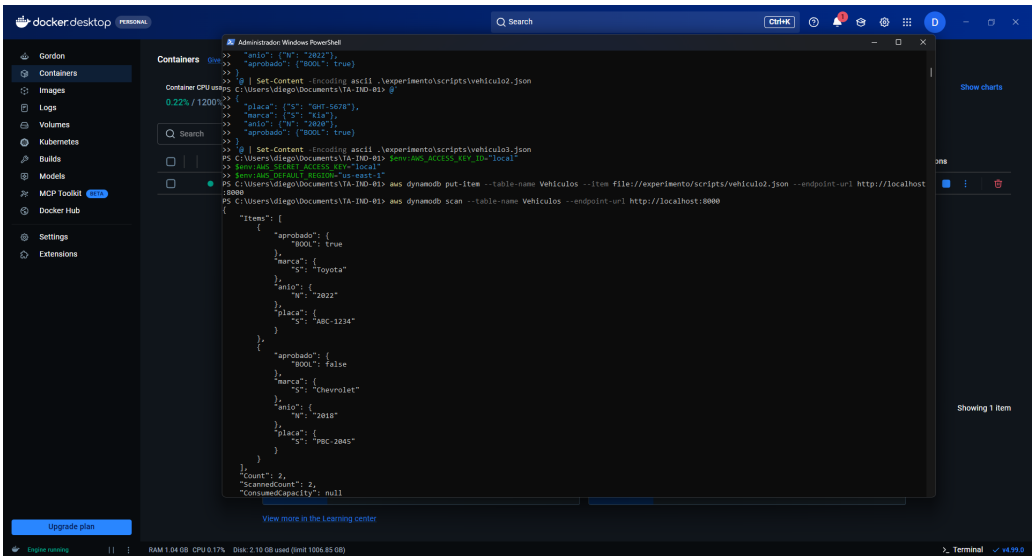


Figura 13: Consulta de registros

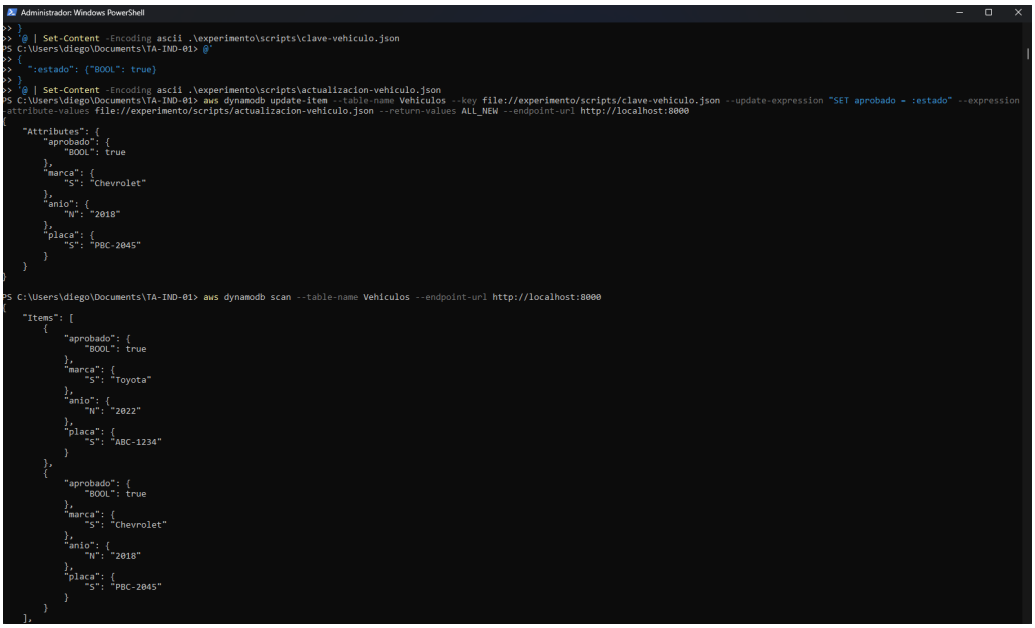


Figura 14: Actualización de un registro

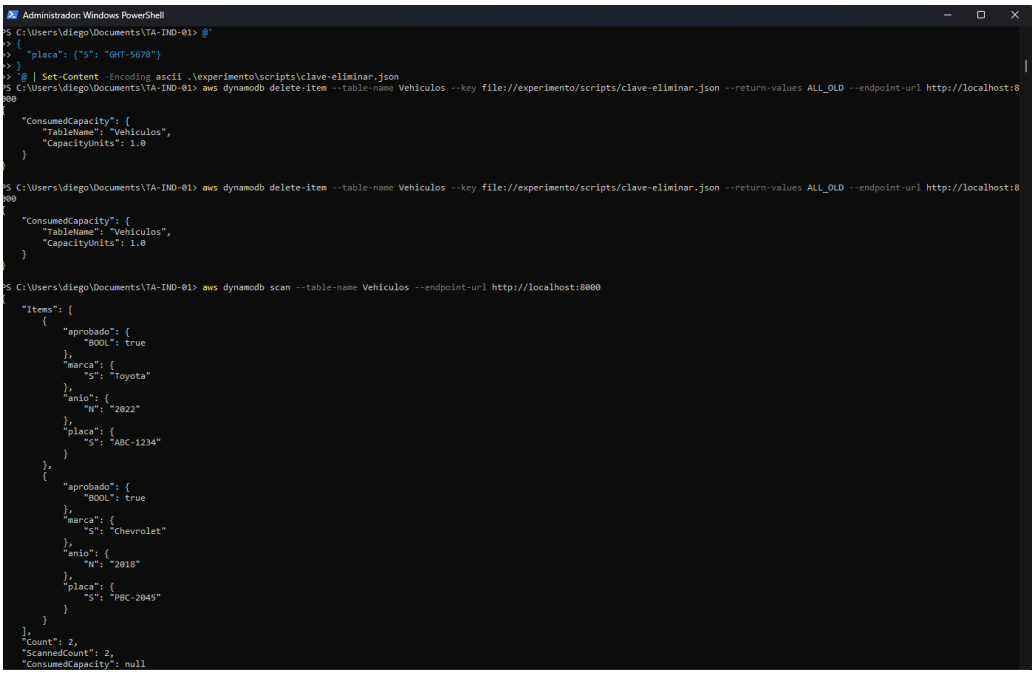


Figura 15: Eliminación de un registro

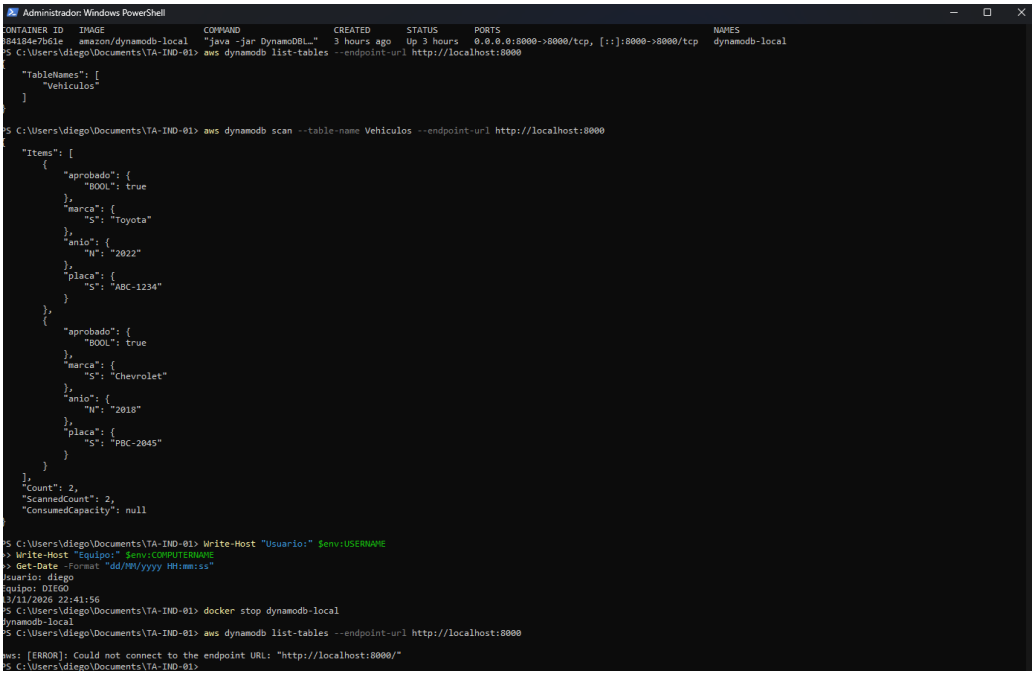


Figura 16: Prueba de tolerancia con el contenedor pausado